University Curriculum Flow and Capacity Planning Using Integer and Binary Programming

Ravi kiran
Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.SC.U4AIE23101

Gopinath
Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.SC.U4AIE23127

Manideep
Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.SC.U4AIE23135

Vamsi Krishna
Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.SC.U4AIE23142

***Abstract*—Universities must decide how many sections (teaching groups) to open for every course before an academic year begins. Opening too few sections leaves students unable to enrol in courses they need; opening too many wastes teaching staff, rooms and timetable slots. In this paper we formulate and implement an Operations Research decision-support framework for university curriculum flow and capacity planning. Using the publicly available StudentDropoutDataset (Zenodo, CC BY 4.0), comprising 464,739 student–course enrolment records covering 39,364 students, 4,989 courses and 163 degree programmes across the academic years 2018, 2021 and 2022, we first project next-year seat demand per course with a validated student- and course-flow model, and then allocate sections with two integer optimisation models. Model 1 is an Integer Programming (IP) formulation that minimises the total number of sections subject to seating all projected demand. Model 2 is a Binary (0–1) Programming formulation of knapsack type that maximises the number of students seated subject to a fixed budget of additional sections, an ordering constraint and an optional per-programme fairness cap. Both models are solved with the HiGHS solver through the SciPy optimisation interface. At a planning capacity of 50 students per section, the flow model projects 161,620 seats of demand for 2023, of which 68 courses in 28 programmes are bottlenecks and 436 students would not obtain a seat. Model 1 returns a proven-optimal allocation of 5,612 sections, a net change of $+42$ sections against the 5,570 currently modelled (67 courses gain sections, 25 release them). Model 2 reduces unmet demand from 436 to 9 students, a 98.0 % reduction, using a budget of only 50 additional sections; the linear-programming relaxation gives a budget shadow price of 0.922 students seated per additional section. The framework is deployed as an interactive Flask web application, verified against a hand calculation and an Excel Solver workbook on a reduced instance, and covered by 32 automated tests.**

***Index Terms*—Curriculum planning, capacity planning, integer programming, binary programming, knapsack problem, operations research, resource allocation, shadow price, HiGHS solver, educational data.**

I. Introduction

University timetabling and capacity planning are resource-allocation problems with a hard integrality structure. A section is an indivisible teaching unit: a department can open three sections of a course or four, but not 3.4. Each section consumes a lecturer, a room and a timetable slot, so sections are scarce and must be justified. Planners must nevertheless guarantee that students can enrol in the courses their degree requires, because a student blocked from a required course is delayed by a full academic term.

Capacity decisions are complicated by the fact that demand for a course is not simply last year's enrolment. Demand has two components: students reaching the course for the first time through normal progression, and students retaking the course after failing it. The second component is driven by course-specific pass rates and is systematically underestimated when planners extrapolate headcounts directly.

Operations Research provides the appropriate machinery [1]. Educational resource allocation has a long literature in automated timetabling [8], which largely addresses the assignment of events to rooms and slots once section counts are known; the complementary question of how many sections to open is the subject of this paper. When the planning budget is unconstrained, the question "what is the smallest number of sections that still seats every student?" is an Integer Programming problem. When the budget is fixed in advance, as it usually is in practice, the question becomes "given B additional sections, where do they do the most good?", which is a Binary (0–1) knapsack problem. The two formulations answer genuinely different managerial questions and we present both.

The principal contributions of this paper are:

1) A reproducible preprocessing and validation pipeline for a public, anonymised university enrolment dataset of 464,739 records, with every published record count independently reproduced.
2) A student- and course-flow model that projects next-year seat demand per course, separating first-time from repeat demand, validated by a split-half procedure at 4.2 % mean absolute percentage error on student stocks and $R^2 = 0.91$ on per-course repeat demand.

3) An Integer Programming formulation (Model 1) that minimises total sections subject to full demand coverage, together with its LP relaxation and integrality gap.
4) A Binary (0–1) Programming formulation (Model 2) of knapsack type that maximises students seated under a fixed section budget, an ordering constraint on successive sections of the same course, and an optional per-programme fairness cap.
5) Duality and sensitivity analysis, including the shadow price of the section budget, the marginal value of each successive section, and sensitivity of the solution to capacity, budget and intake growth.
6) A three-way verification protocol in which the same reduced instance is solved by hand, in Excel Solver and by the deployed application, with agreement required.
7) Deployment as an interactive Flask web application with a dashboard, dataset explorer, a page per model and a model-comparison page, supported by 32 automated tests.

Fig. **??** shows the end-to-end pipeline, from the published records through demand projection and bottleneck diagnosis to the two optimisation models and the delivered artefacts.

A deliberate scope restriction applies throughout. Section capacity is a *planning parameter chosen by the university*, not a variable observed in the dataset; the published data contains no room or capacity field. Furthermore, this work does *not* predict which individual students will drop out. The models allocate teaching capacity; they do not profile students.

II. Study Data and Planning Parameters

A. Empirical Database Overview

The empirical foundation is the StudentDropoutDataset published on Zenodo under the Creative Commons Attribution 4.0 licence [**?**]. The archive contains student–course enrolment records from a Spanish public university for the academic years 2018, 2021 and 2022. Identifiers for students, courses and programmes are published as irreversible hashes; no personally identifying information is present. Hashes serve only as join keys, so they constrain neither the arithmetic nor the interpretation of the models.

B. Preprocessing and Count Validation

Records are cleaned, typed and linked into a student–course table and a student–year table. Because the Zenodo record publishes no column codebook, every derived field is validated against the record counts stated in the dataset description. Table I reports this reconciliation; all five published counts are reproduced exactly, which is the evidence that the column interpretation is correct.

Table II summarises the cleaned data by academic year. The pass rate declines slightly across the observed years, from 73.9 % in 2018 to 71.7 % in 2022. In 2022, 13,446 enrolments (8.4 % of all enrolments) were retakes, and 5,474 students (26.8 % of the student body) retook at least one course. Repeat enrolment is therefore a material component of demand and is modelled explicitly.

TABLE I
Dataset validation against published record counts

Quantity	Expected	Observed	Match
Enrolment records	464,739	464,739	Yes
Students	39,364	39,364	Yes
Courses	4,989	4,989	Yes
Programmes	163	163	Yes
Academic years	3	3	Yes

TABLE II
Descriptive statistics by academic year

Year	Records	Students	Courses	Pass rate	Mean grade
2018	152,446	20,233	3,747	73.9 %	6.39
2021	153,120	20,264	3,859	72.9 %	6.41
2022	159,173	20,427	3,787	71.7 %	6.34

C. Planning Parameters

The parameters in Table III are assumptions of the planning model. They are set by the planner and exposed as controls in the web application; none of them is read from the dataset.

TABLE III
Planning parameters (model assumptions, not dataset variables)

Symbol	Parameter	Value
cap	Section capacity (students per section)	50
g	First-time demand growth per year	0 %
B	Additional sections available (Model 2)	50
M_p	Maximum extra sections per programme	5
θ	Bottleneck threshold (demand / capacity)	1.00
T	Planning horizon for scenarios (years)	5

Because the dataset contains no capacity field, the number of sections currently operating is itself modelled. For each course c the current section count is taken as

$$s_c = \left\lceil \frac{E_c}{cap} \right\rceil, \tag{1}$$

where E_c is observed 2022 enrolment. At $cap = 50$ this yields 5,570 sections in total, which is the baseline against which both models are compared.

D. Demand Projection

Demand is projected with a stock-and-flow formulation in the system-dynamics tradition [9], in which students accumulate in year-level stocks and move between them at estimated rates.

Let F_c denote projected first-time demand for course c, ρ_c the course-specific retake rate estimated by empirical-Bayes shrinkage toward the university-wide rate, and R_c the pool of students eligible to retake. Projected demand for the planning year is

$$D_c = F_c(1 + g) + \rho_c R_c. \tag{2}$$

Applying (2) with $g = 0$ gives a total projected demand of 161,620 seats for 2023, decomposing into 145,727 first-time seats and 15,893 repeat seats.

The projection is validated by a split-half procedure: flow rates are estimated on one random half of the student population and used to predict the other half. The mean absolute percentage error across the student-flow stocks is 4.2 %. For per-course repeat demand, course-specific retake rates achieve $R^2 = 0.909$ with a mean absolute error of 0.77 students per course, against $R^2 = 0.743$ for a single university-wide rate. The course-specific estimator carries a small upward bias of $+6.8\,\%$ in total, which is conservative for a capacity-planning application because it errs toward providing seats.

III. Mathematical Formulations

Let $i \in \{1, \ldots, n\}$ index the $n = 3{,}787$ courses offered in the planning year, with projected demand D_c from (2) and section capacity cap.

A. Model 1: Minimum Section Allocation (Integer Programming)

Decision variable. Let $x_c \in \mathbb{Z}_+$ denote the number of sections opened for course c [3]. Sections are indivisible, which is what makes this an integer programme rather than a linear one.

Objective. Minimise the total number of sections opened:

$$\min \; Z_1 = \sum_{c=1}^{n} x_c. \tag{3}$$

Constraints.

$$cap \cdot x_c \geq (1+\beta) D_c, \qquad c = 1, \ldots, n, \tag{4}$$

$$x_c \geq 1, \qquad \forall\, c : D_c > 0, \tag{5}$$

$$x_c \in \mathbb{Z}_+, \qquad c = 1, \ldots, n. \tag{6}$$

Constraint (4) is the service guarantee: the seats provided must cover projected demand, optionally inflated by a safety buffer $\beta \geq 0$ (set to zero in the reported results). Constraint (5) ensures that any course with positive demand actually runs.

A structural property of this formulation is worth stating because it has practical consequences. Each constraint in (4) involves only its own variable x_c, so the problem *separates by course* and the optimum has the closed form

$$x_c^{\star} = \max\left(1, \left\lceil \frac{(1+\beta) D_c}{cap} \right\rceil \right). \tag{7}$$

Model 1 is therefore solvable by hand, which we exploit in Section V-G as an exact check on the solver. Model 2 has no such property.

B. Model 2: Allocation of Limited Additional Capacity (Binary Programming)

In practice the number of extra sections that can be funded is fixed in advance. Let s_c be the sections currently operating and define the unmet demand of course c as

$$U_c = \max(0,\; D_c - cap \cdot s_c). \tag{8}$$

Decision variable. For course c, let $y_{ck} \in \{0, 1\}$ equal 1 if the k-th additional section is opened for that course and 0 otherwise, for $k = 1, \ldots, K_c$ with $K_c = \lceil U_c / cap \rceil$. Each candidate section is an item and the budget is the knapsack [4].

Coefficients. The k-th additional section of course c seats

$$a_{ck} = \min\big(cap,\; U_c - (k-1)\, cap\big) \tag{9}$$

students, which is a diminishing quantity: the last section opened for a course is usually only partly filled. Let $w_c \geq 1$ be a course priority weight (Section III-D); $w_c = 1$ for all c in the reported results.

Objective. Maximise the weighted number of students seated:

$$\max \; Z_2 = \sum_{c=1}^{n} \sum_{k=1}^{K_c} w_c\, a_{ck}\, y_{ck}. \tag{10}$$

Constraints.

$$\sum_{c=1}^{n} \sum_{k=1}^{K_c} y_{ck} \leq B, \tag{11}$$

$$y_{ck} \leq y_{c,k-1}, \quad k = 2, \ldots, K_c, \tag{12}$$

$$\sum_{c \in p} \sum_{k=1}^{K_c} y_{ck} \leq M_p, \quad \forall\, \text{programmes } p, \tag{13}$$

$$y_{ck} \in \{0, 1\}. \tag{14}$$

Constraint (11) is the section budget. Constraint (12) forces sections of a course to be opened in order, which prevents the solver from claiming the high coverage of a first section while nominally opening a later one. Constraint (13) is an optional fairness cap limiting how many extra sections any one programme may absorb. Maximising coverage in (10) is equivalent to minimising total residual unmet demand $\sum_c U_c - \sum_{c,k} a_{ck} y_{ck}$.

Unlike Model 1, Model 2 does *not* separate by course: all courses compete for the shared budget in (11), and the fairness cap couples courses within a programme. This coupling is what genuinely requires an optimisation solver.

C. LP Relaxation and Sensitivity Analysis

Relaxing $y_{ck} \in \{0, 1\}$ to $0 \leq y_{ck} \leq 1$ yields a linear programme whose dual provides managerial information [2]. The shadow price of (11) is the number of additional students seated by one more section of budget. Because each $y_{ck} \leq 1$, the objective is piecewise linear in B with a breakpoint at every whole section, so the ranging table reports the marginal value of each successive section. The LP relaxation of Model 1 is likewise solved to obtain a lower bound on (3) and hence the integrality gap.

D. Course Priority Weights via AHP

When the planner wishes to express a priority other than raw headcount, the weights w_c in (10) may be derived by the Analytic Hierarchy Process [5] over three criteria: the repeat-student share of a course's demand, whether the course

is a first-year course, and one minus its pass rate. For a representative pairwise comparison matrix the resulting criteria weights are 0.648, 0.230 and 0.122 respectively, with a consistency ratio of $CR = 0.003$, well inside the conventional $CR \leq 0.10$ acceptance threshold. The course weight is then $w_c = 1 + \sum_k W_k \cdot \text{score}_k(c)$.

IV. System Architecture and Implementation

The system is implemented in Python 3.13 with a modular architecture that separates the analytical core from the interface:

- `preprocessing.py` — loads the published archive, cleans and links the student–course tables, and writes the count-validation log.
- `system_dynamics.py` — student- and course-flow models, empirical-Bayes rate shrinkage, and the split-half validation routines.
- `capacity.py` — modelled current sections, bottleneck classification and bottleneck summaries.
- `optimization.py` — constructs the constraint matrices for both models and calls `scipy.optimize.milp` [7], which uses the HiGHS branch-and-bound and dual-simplex solvers [6]; extracts primal solutions, slacks and duals.
- `mcdm.py` — AHP pairwise comparison, principal eigenvector weights, and the consistency ratio.
- `scenarios.py` — couples the flow model and the optimisation models in a multi-year loop, with shortfall returning as next-year backlog, and performs the sensitivity sweeps.
- `run_pipeline.py` — batch execution writing all tables, figures and the auto-generated results report.
- `flask_app.py` — Flask web application with a dashboard, a dataset explorer providing statistics on demand, a page for each optimisation model, and a model-comparison page; charts are rendered with Chart.js.
- Test suite — 32 automated tests covering unit behaviour of the formulations and end-to-end assertions against the real dataset.

Heavy computation (preprocessing, flow calibration, demand projection) is performed once by the pipeline and cached; the web application re-solves both optimisation models live for any parameter setting, which completes in well under one second for the full 3,787-course instance.

V. Results and Discussion

A. Baseline and Bottleneck Diagnosis

At $cap = 50$ the modelled current provision is 5,570 sections, giving 278,500 seats against 161,620 seats of projected demand. Aggregate capacity is therefore ample; the problem is distributional. Of 3,787 courses, 68 are bottlenecks ($D_c \geq \theta \cdot cap \cdot s_c$) and a further 267 are at risk at the 90 % utilisation threshold. The bottlenecks span 28 degree programmes and leave 436 students without a seat. Repeat enrolment accounts for 16.4 % of demand in bottleneck courses, against 9.8 % across all courses, confirming that retakes concentrate in exactly the courses that are already full.

B. Model 1: Minimum Section Allocation

Model 1 returns a proven-optimal allocation of 5,612 sections (HiGHS status: optimal; all variables integral; demand fully met). Against the 5,570 modelled current sections this is a net change of only $+42$: 67 courses require additional sections and 25 courses can release a section without breaching (4). The managerial reading is that the institution is not short of teaching capacity in aggregate but is misallocating it, and that full coverage is achievable at close to budget neutrality.

The LP relaxation of (3)–(6) attains 5,062.3 fractional sections, so the integrality gap is 549.7 sections. The gap is large because (5) forces a whole section onto every small course; this also explains the 57.6 % mean seat utilisation of the integer solution, which is a consequence of indivisibility rather than of over-provision.

C. Model 2: Limited Additional Capacity

With a budget of $B = 50$ additional sections and uniform weights, Model 2 seats 427.7 of the 436.5 unmet students, reducing unmet demand to 8.8 students, a 98.0 % reduction. The number of courses still short falls from 67 to 17. All 50 sections are used, each going to a distinct course. Clearing the shortfall entirely requires 67 sections. Table IV compares the baseline against both models.

TABLE IV
System performance comparison: baseline vs. Model 1 vs. Model 2

Evaluation metric	Baseline	Model 1	Model 2
Sections operated	5,570	5,612	5,620
Change vs. baseline	—	+42	+50
Students without a seat	436.5	0.0	8.8
Reduction in unmet demand	—	100.0 %	98.0 %
Courses still short	67	0	17
Seats provided	278,500	280,600	281,000
Mean seat utilisation	58.0 %	57.6 %	57.5 %
Solver status	—	Optimal	Optimal

The two models answer different questions and neither dominates. Model 1 describes the ideal allocation when the budget is open; Model 2 describes the best achievable outcome when it is not. Model 2 attains 98 % of the available benefit with a budget 25 % smaller than the 67 sections that full coverage requires, which is the practically relevant finding.

D. Duality, Binding Constraints and Shadow Prices

The LP relaxation of Model 2 yields the following dual information at $B = 50$:

- **Section budget.** The budget constraint (11) is strictly binding (50 of 50 sections used). Its shadow price is **0.922 students per section**: one additional section of budget seats approximately 0.92 further students at this point in the ranging table.

- **Marginal value of successive sections.** The 50th section is worth 0.938 students and the 51st is worth 0.922, confirming diminishing returns. Early sections are far more valuable: the first five sections seat 30.3 students each on average, falling to 15.1 at $B = 10$, 8.4 at $B = 25$ and 3.0 at $B = 50$.
- **Saturation.** Beyond $B = 67$ the shadow price falls to zero. No further section can seat anyone because all unmet demand has been absorbed; additional budget is pure slack.
- **Fairness cap.** Imposing $M_p = 5$ extra sections per programme raises residual unmet demand from 8.8 to 19.6 students. The cost of the equity constraint is therefore 10.8 students, which quantifies the efficiency–equity trade-off explicitly rather than leaving it to judgement.

E. Sensitivity Analysis

Table V reports sensitivity to the section budget and Table VI to the capacity parameter.

TABLE V
SENSITIVITY TO THE ADDITIONAL-SECTION BUDGET (MODEL 2)

Budget B	Sections used	Unmet after	Marginal coverage
0	0	436.5	—
5	5	285.0	30.29
10	10	209.4	15.11
25	25	83.9	8.37
50	50	8.8	3.00
75	67	0.0	0.35
100	67	0.0	0.00

TABLE VI
SENSITIVITY TO THE SECTION-CAPACITY PARAMETER

cap	Bottlenecks	At risk	Model 1 sections	Utilisation
30	133	472	7,533	71.5 %
40	86	345	6,278	64.4 %
50	68	267	5,612	57.6 %
60	48	206	5,200	51.8 %
75	38	202	4,752	45.3 %
100	31	128	4,337	37.3 %

Larger sections reduce both the number of bottlenecks and the number of sections required, but at a monotonic cost in seat utilisation and, in practice, in teaching quality. The capacity parameter is therefore a genuine policy lever rather than a technical constant, which is precisely why it is exposed in the interface.

A caveat applies to Table VI. Status-quo unmet demand is *not* monotone in *cap*, because the modelled baseline $s_c = \lceil E_c/cap \rceil$ is itself recomputed at each capacity. Rounding effects therefore move the baseline as the parameter moves. The Model 1 column, which does not depend on the baseline, is monotone as expected.

F. Multi-Year Scenario Analysis

Coupling the flow model with each allocation policy over a five-year horizon, with any shortfall returning as next-year backlog, gives Table VII.

TABLE VII
SCENARIO COMPARISON, 2023–2027

Scenario	Policy	Year-1 unserved	Cumulat
S0 Status quo	none	436.5	3,38
S1 Limited extra	Model 2, B=50	8.8	9
S2 Double budget	Model 2, B=100	0.0	
S3 Full coverage	Model 1	0.0	
S4 Model 2 + pass +5pp	Model 2	8.8	1
S5 Model 2, intake +5%/yr	Model 2	354.9	29,34

Scenario S5 is the substantive result. A sustained 5 % annual growth in first-time intake drives cumulative unserved demand to 29,344 students over five years despite the same 50-section intervention that is almost sufficient under flat intake. No realistic section budget absorbs this: the binding issue becomes intake policy rather than timetable capacity. Conversely, S4 shows that a five-percentage-point improvement in pass rates reduces cumulative unserved demand from 94.5 to 12.8, because fewer retakes are generated, and simultaneously reduces the terminal backlog from 46,348 to 37,454. Academic support and capacity planning are complementary interventions.

G. Three-Way Verification

To validate the implementation independently of the solver, a reduced instance of the eight largest bottleneck courses was solved by three methods: by hand, in Microsoft Excel Solver (Simplex LP with integer and binary constraints), and by the deployed application. Table VIII reports the outcome. Agreement is exact.

TABLE VIII
THREE-WAY VERIFICATION ON THE EIGHT-COURSE INSTANCE

Model	Method	Result
Model 1	Hand, eq. (7)	48 sections
Model 1	Excel Solver (integer)	48 sections
Model 1	Application (HiGHS)	48 sections
Model 2	Hand, B=3, no cap	112.22 students
Model 2	Hand, B=3, M_p=1	88.62 students
Model 2	Application (HiGHS), M_p=1	88.62 students

The Model 2 rows demonstrate the non-separability claimed in Section III. Without the fairness cap the optimum is obtained by selecting the three courses with the largest unmet demand. With $M_p = 1$ that greedy selection becomes infeasible, the optimal selection changes, and the objective falls by 23.60 students. A greedy heuristic would return the wrong answer; the binary programme does not.

H. Effect of Priority Weights

An honest negative result should be reported. Substituting repeat-weighted priorities for uniform weights in (10) leaves

the solution unchanged at $B = 50$ (8.8 students unmet in both cases), and AHP-derived weights likewise. The coverage coefficients a_{ck} dominate the weight multipliers at this budget, so the weighting scheme is expressive but not, on this dataset, decisive. We report the AHP machinery and its consistency check as a methodological component rather than as a driver of the result.

VI. Limitations

1) **Capacity is assumed homogeneous.** Every section is assumed to seat cap students. Real rooms differ in size, and laboratory or studio courses have different ratios. The model treats capacity as a single policy parameter.
2) **No timetable feasibility.** The models determine how many sections to open, not when they meet or who teaches them. A section count that is feasible in aggregate may be infeasible once room and staff availability are imposed.
3) **Deterministic demand.** Projected demand is a point estimate. Stochastic variation in enrolment is not propagated into the allocation, although the safety buffer β provides a deterministic proxy.
4) **Non-consecutive observation years.** The archive provides 2018, 2021 and 2022, so only the 2021–2022 pair is consecutive and available for flow calibration; the 2018 cohort informs descriptive statistics only.
5) **Undocumented schema.** The Zenodo record publishes no column codebook. Column interpretation rests on the exact reproduction of all published record counts (Table I) and on internal consistency checks.

VII. Conclusion and Future Work

This paper presented an Operations Research decision-support system for university curriculum flow and capacity planning. Using 464,739 enrolment records from a public, anonymised dataset, a validated flow model projects 161,620 seats of demand for the planning year, of which 68 courses are bottlenecks leaving 436 students without a seat.

An Integer Programming formulation establishes that full coverage requires 5,612 sections, a net increase of only 42 over the 5,570 currently modelled, because 67 courses gain sections while 25 release them. A Binary (0–1) Programming formulation establishes that a fixed budget of 50 additional sections removes 98.0 % of the shortfall, reducing unmet demand from 436 to 9 students, with a budget shadow price of 0.922 students per section and complete saturation at 67 sections. Imposing a per-programme fairness cap costs 10.8 students, quantifying the equity–efficiency trade-off. Both models solve to proven optimality and agree exactly with hand and Excel Solver computations on a reduced instance.

The scenario analysis qualifies the recommendation. Under flat intake the proposed intervention is close to sufficient, but a sustained 5 % annual growth in first-time intake generates 29,344 cumulative unserved students over five years, which no plausible section budget absorbs. Capacity optimisation is necessary but not sufficient; it must be paired with intake planning and with measures that raise pass rates and so reduce retake demand.

Future work will extend the formulation in three directions: a two-stage stochastic programme in which the second-stage recourse reallocates sections after observed enrolment; a multi-period model in which section decisions in one year constrain the next through staff contracts; and integration of room-level capacity and timetable feasibility so that the allocation produced is directly implementable.

Reproducibility

The dataset is publicly available under CC BY 4.0 [**?**]. All results in this paper are regenerated end to end by a single command and are accompanied by 32 automated tests. Source code and generated outputs are available at https://github.com/gopinathgatta/University-Curriculum-Flow-and-Capacity-Planning-Using-Integer-and-B

References

[1] F. S. Hillier and G. J. Lieberman, *Introduction to Operations Research*, 11th ed. New York, NY, USA: McGraw-Hill Education, 2021.
[2] G. B. Dantzig, *Linear Programming and Extensions*. Princeton, NJ, USA: Princeton University Press, 1963.
[3] G. L. Nemhauser and L. A. Wolsey, *Integer and Combinatorial Optimization*. New York, NY, USA: Wiley-Interscience, 1988.
[4] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. Chichester, U.K.: John Wiley & Sons, 1990.
[5] T. L. Saaty, "How to make a decision: The Analytic Hierarchy Process," *European Journal of Operational Research*, vol. 48, no. 1, pp. 9–26, 1990.
[6] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.
[7] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
[8] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266–280, 2002.
[9] J. W. Forrester, *Industrial Dynamics*. Cambridge, MA, USA: MIT Press, 1961.